

What is SDLC, Why and where it is used and how to implement it.

SDLC : software developer life cycle

Steps :

Req Gathering, Design, coding& implementation , testing, deploy , maintenance.

Requirement Gathering – Functional, Non- Functional

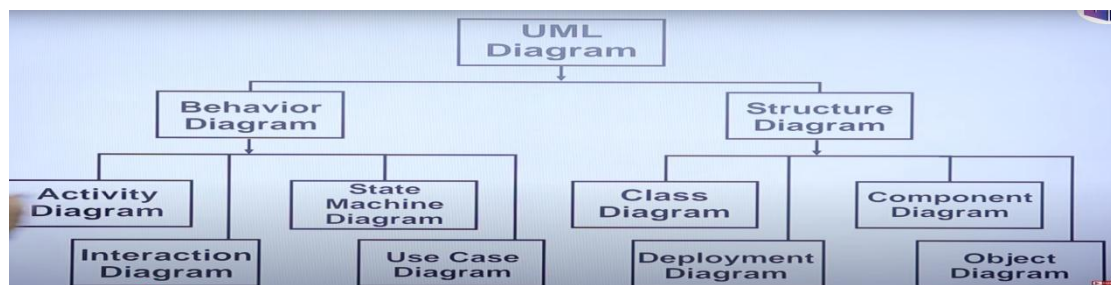
- Requirements, which are related to functional/Working aspect of software fall into this category.
- Non-Functional Requirements are expected characteristics of target software. (Security, Storage, Configuration, Performance, Cost, Interoperability, Flexibility, Disaster recovery, Accessibility)

System behaviour : functional req

Design- UML,

Structural : Static (Actual)

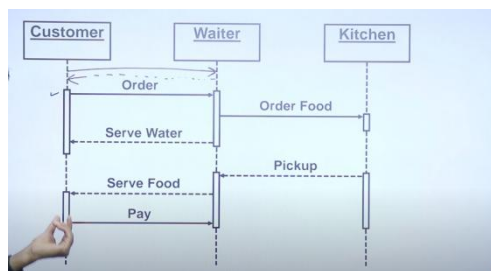
Behavioural : Dynamic (Activity)



Sequence Diagram :

Object ,

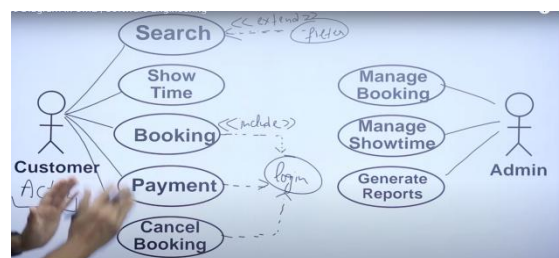
Activation time



Use Case Diagram :

Actor (customer, Admin),

System boundary , include , exclude



Waterfall → Requirements are fixed and clear (e.g., banking).

V-Model → When safety and testing are top priority (e.g., medical software).

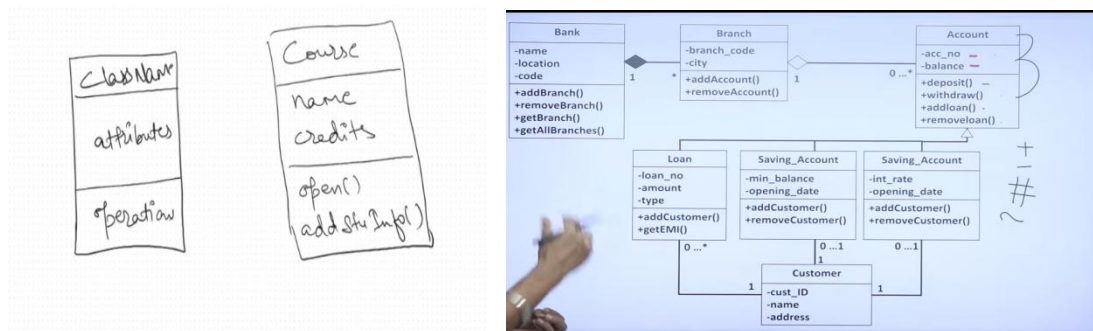
Spiral → High-risk, large, and costly projects (e.g., NASA projects).

Iterative → Products that evolve with time (e.g., CRMs).

Agile → Startups, dynamic environments, frequent changes (e.g., mobile apps) (Incremental ,iterative),**"Adaptive"**

Incremental	Iterative
Each part complete building then move forward	In cycle way we get product
Add new features one by one	Same product, keep improving
Deliver functional part over time	

Class Diagram :



Software Testing Life Cycle

Ensure that the software product satisfies the **functional and non-functional requirements** defined in the SRS (Software Requirement Specification).

1. Testing Shows Presence of Defects

- Testing can show that **defects are present**, but it **cannot prove there are no defects**.
- Even multiple testing doesn't prove there is 100% bug-free.

2. Exhaustive (all testing) Testing is Impossible

Testing all possible inputs, scenarios, and paths is **impractical**, so **risk-based testing** (only test some test cases) is used.

3. Early Testing

Start testing as **early as possible** in the SDLC to **reduce cost and time**.

4. Pesticide Paradox

Repeating the same tests will **not find new bugs**. Test cases should be **reviewed and updated**.

5. Testing is Context Dependent

Different applications require **different types of testing** (e.g., banking app vs. game app).

6. Absence of Errors Fallacy

Finding and fixing defects does **not guarantee success** if the product does not meet **business needs**.

Defect Clustering

Most defects are found in a **few modules** (Pareto principle: 80% defects in 20% modules).

Testing process:

Test Plan: A **formal document** describing the **strategy, objectives, resources, scope, and schedule** of testing activities.

Test Scenario: it is basically complete test description .(High level documentation)

It represent feature to be tested.

Test Cases: A **detailed set of steps** to validate a particular feature or scenario.(Test Case ID ,Expected Result,Actual Result,Status (Pass/Fail))

(Low level /Detailed documentation)

Test Suite: A **collection of test cases** grouped together for a specific module or functionality.

Testing Level:

✓ **Unit Testing** – Testing individual components or modules of the software in isolation.

✓ **Integration Testing** – Testing the interaction and data flow between combined modules.

✓ **System Testing** – Testing the complete integrated system to ensure it meets requirements.

✓ **Acceptance Testing** – Testing done by end-users or clients to validate business needs before release.

- **Alpha Testing** – At developer site.
- **Beta Testing** –end user or last phase people.

Regression Testing : Tests **existing features** after new code changes to ensure nothing is broken.

✓ **Functional Testing** – Validates that the software **functions according to specified requirements**.

✓ **Non-Functional Testing** – Validates the **quality attributes** of the software such as performance, usability, security, and reliability.

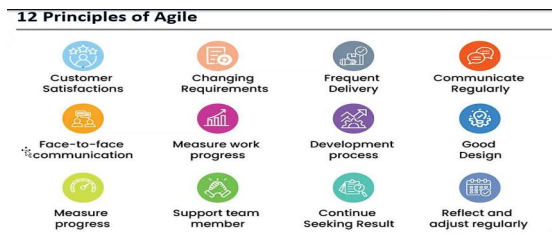
Testing Techniques:

- ✓ **Black-box Testing** – Testing the software **without knowing the internal code**, focusing only on inputs and outputs.
- ✓ **White-box Testing** – Testing the software **with full knowledge of the internal code and logic**.
- ✓ **Grey-box Testing** – Testing the software **with partial knowledge of the internal code and design**.

Test Lifecycle:

- ✓ **Planning** – Define the **scope, strategy, resources, and schedule** for testing activities.
- ✓ **Design** – Create **test scenarios, test cases, and prepare test data** based on requirements.
- ✓ **Execution** – Perform **test cases, log results, and compare actual vs expected outcomes**.
- ✓ **Defect Reporting** – Document and report **any defects or issues** found during testing. (Logging and tracking issue).
- ✓ **Closure** – Conclude testing by **analyzing results, preparing reports, and signing off** after goals are met.

Agile & Scrum



✓ What is Agile?

Agile is a **software development methodology** that emphasizes **iterative development, collaboration, customer feedback**, and **flexibility to change**. Instead of delivering the entire product at the end, Agile delivers small, functional increments frequently.

4 Core Values (Pillars):

- **Individuals and interactions** over processes and tools
- • **Working software** over comprehensive documentation
- • **Customer collaboration** over contract negotiation
- • **Responding to change** over following a plan

Scrum : <https://youtu.be/xVi3lgyD080?si=ftc1NWgAQ9ORqXbQ>

✓ What is Scrum?

Scrum is a **lightweight, iterative, and incremental Agile framework** used to develop and deliver complex products in small, manageable increments called **Sprints(Time Box)**.

- **Sprint Duration:** Usually **1 to 4 weeks**.
- **Goal:** Deliver a **potentially shippable product increment** at the end of each Sprint.

Scrum workflow :

- **Product Owner :** Maintains the **Product Backlog** (list of requirements).
- **Product Backlog:** list of requirements, features get in product.
- **Sprint Planning :** product owner, Scrum master doing meeting which specific feature implement within small time.
- **Sprint Backlog:** small list of requirements, from product backlog.
- **Development & Daily Scrum :** Build increment.
- **Sprint Review** → Show increment to stakeholders.
- **Sprint Retrospective** → Improve process for next Sprint.

- **Roles & Responsibilities:** Product Owner, Scrum Master, Scrum / Development Team, Stakeholders & Customers
- **Agile Ceremonies:** Sprint Planning, Daily Stand-ups, Sprint Review, Sprint Retrospective, Backlog Refinement
- **Agile Artifacts:** Product Backlog, Sprint Backlog, Increment, Definition of Done (DoD), Burn-down Charts

✓ Agile Practices (with example in 1 line)

Epic – A large body of work that can be broken down into smaller deliverables (features and user stories).

Example: “Create an online shopping app.”

Feature – A major part of an Epic.(A big useful function inside an app)

Example: “Add payment system.”

User Story – A small user need in simple words.

Example: “As a user, I want to pay by credit card.”

Task – A small technical step to complete a story.

Example: “Design payment page.”

Bug – A problem or error in the product.

Example: “Payment button not working.”

Acceptance Criteria – Rules to say work is complete.

Example: “Payment should work for Visa and MasterCard.”

✓ Simple Definition:

A **code smell** is **bad programming practice** that signals your code needs improve.

✓ Examples of Code Smells:

Duplicated Code – Same logic repeated in many places.

Example: Copy-pasting the same validation logic in multiple functions.

Long Method – A function is too long and does too many things.

✗ Example: A 200-line `processOrder()` function instead of smaller ones.

Large Class – A single class handles too many responsibilities.

✗ Example: A `UserManager` class that manages login, payments, and reports.

Dead Code – Unused variables, methods, or classes.

✗ Example: A method written but never called.

Long Parameter List – A function takes too many arguments.

✗ Example: `calculatePrice(item, qty, tax, discount, customerType, region, date, ...)`

CI/CD (Continuous Integration and Continuous Deployment/Delivery)

Continuous Integration (CI): Developers frequently integrate (merge) their code into a shared repository.

EXAMPLE: GitHub Actions, Jenkins, GitLab CI, CircleCI

Continuous Delivery: Code changes are automatically tested and prepared for release to production (manual approval before deploy).

Continuous Deployment: Fully automated; every change that passes tests is deployed directly to production.

EXAMPLE : Docker, Kubernetes, AWS CodePipeline, ArgoCD

Method :

✓ 1. Unit Testing

Definition: Testing individual parts (functions, methods) of code to ensure they work correctly.

Example: Checking if a `sum(2, 3)` function returns 5.

✓ 2. Static Code Analysis

Definition: Analyzing code **without running it** to find bugs, security issues, and code smells.

Example: Using **SonarQube** to detect unused variables or vulnerabilities in code.

✓ 3. Code Quality / Vulnerability (weakness in software)

Definition: Measuring how clean, readable, and secure your code is; finding potential security risks.

Example: Checking if passwords are stored in plain text (bad practice).

✓ 4. Automation

Definition: Using tools to run tests automatically instead of manually.

Example: **Selenium** clicks all buttons on a web app to check if they work, without human effort.

✓ 5. Reports

Definition: Summarized results of tests, analysis, and deployment for developers and managers.

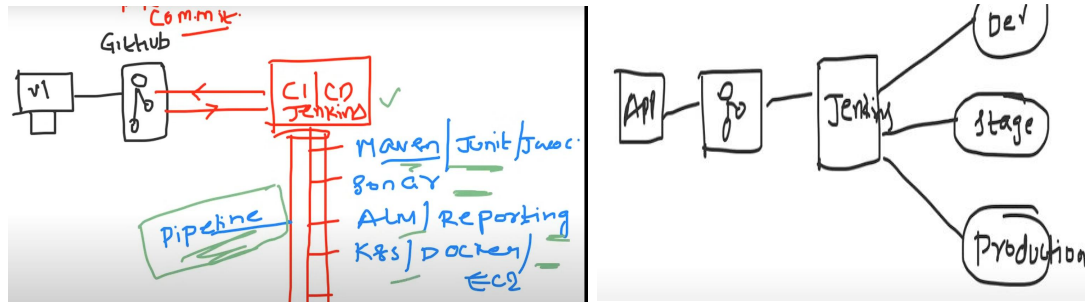
Example: A **JUnit test report** showing which test cases passed and failed.

✓ 6. Deployment

Definition: Moving your application from development to production for users to access.

Example: Deploying a web app to **AWS or Heroku** so customers can use it.

<https://youtu.be/CmVxoNkkACQ?si=Q1bG2d8R6yIpMSOS>



Feature	Traditional Model	CI/CD Model
Testing	At end, manual	Automated, after every code push
Deployment	Manual, infrequent	Automated, frequent (daily/hourly)
Release Speed	Slow (months)	Fast (daily/weekly)
Bug Detection	Late	Early (in every build)
Risk	High (big releases)	Low (small, frequent updates)

✓ 1. Code Commit

Definition: Developers push (commit) code changes to a shared repository like **GitHub/GitLab**.

Example:

`git push origin main` → sends code to GitHub.

✓ 2. Build

Definition: Converts source code into an executable or deployable format.

Example:

Compiling Java files into `.class` files, creating a `.jar` or Docker image.

✓ 3. Test

Definition: Runs automated tests to verify functionality, security, and performance.

Example:

Unit tests check if the **login function** works after changes.

✓ 4. Deploy

Definition: Sends the tested code to a staging or production environment.

Example: Deploying a web app to **AWS, Azure, or Heroku** automatically.

✓ 5. Monitor

Definition: Tracks application performance and errors after deployment **Example:**

Using **Prometheus** or **New Relic** to monitor server health and error logs.

✓ In short:

CI/CD Pipeline automates:

Code → Build → Test → Deploy → Monitor